Hugo, Podman & Caddy Workflow

Spickzettel & Befehlsübersicht für Fedora

Stephan

10. August 2026

Zusammenfassung

Eine kompakte Übersicht über den lokalen Entwicklungs-Workflow mit Hugo, Multi-Stage Container-Builds und automatisiertem PDF-Export.

Inhaltsverzeichnis

asciinema rec und play vorbereiten . . . 1
login dockerhub . . . 1
login github . . . 1
gemini chat . . . 2
asciinema command . . . 2
Vorraussetzungen . . . 3
System Details Report . . . 3
Report details . . . 3
Hardware Information: . . . 3
Software Information: . . . 3

Hugo Container Workflow (Fedora 43 / Podman) 4
1. Containerfile für die lokale Entwicklung . . . 4

asciinema rec und play vorbereiten

- Quelle: https://github.com/asciinema/asciinema
- `$ asciinema rec 1.cast`
- ‘[ctrl] + [D]
- `$ asciinema play 1.cast`

login dockerhub

- Login docker
- `$ podman login docker.io -u stephanbac`
- Passwort eingabe

login github

- gh command?
- gh gist command?

gemini chat

---

asciinema command

```
which asciinema
```

```
/usr/bin/asciinema
```

Danach

```
whatis asciinema
```

```
asciinema (1)        - Terminal session recorder, streamer, and player
```

```
whereis asciinema
```

```
Aasciinema: /usr/bin/asciinema /usr/share/man/man1/asciinema.1.gz
```

```
tldrg asciinema
```

```
:r !bash -i -c 'tldrg asciinema'
```

```
in vim
```

- Zeichnen Sie Terminalsitzungen auf, geben Sie sie wieder und teilen Sie sie optional auf https://asciinema.org. Siehe auch: „Terminalizer“, „agg“.
- Weitere Informationen: https://docs.asciinema.org/manual/cli/.
- Verknüpfen Sie die lokale Installation von [3masciinema [23m mit einem asciinema.org-Konto: asciinema auth
- Erstellen Sie eine neue Aufnahme und speichern Sie sie in einer lokalen Datei (beenden Sie sie mit [3m [23m oder geben Sie [3mexit [23m] ein): asciinema record path/to/recording.cast
- Spielen Sie eine Terminalaufzeichnung aus einer lokalen Datei ab: asciinema play path/to/recording.cast
- Spielen Sie eine auf https://asciinema.org gehostete Terminalaufzeichnung ab: asciinema play https://asciinema.org/a/cast_id
- Machen Sie eine neue Aufnahme und begrenzen Sie die Leerlaufzeit auf höchstens 2,5 Sekunden: asciinema record –idle-time-limit 2.5
- Drucken Sie die vollständige Ausgabe einer lokal gespeicherten Aufnahme aus: asciinema cat path/to/recording.cast
- Laden Sie eine lokal gespeicherte Terminalsitzung auf asciinema.org hoch: asciinema upload path/to/recording.cast
- Streamen Sie das aktuelle Terminal auf einer lokalen Webseite: asciinema stream –local

```
:r !man asciinema | col -b | trans en:de -b
```

```
in vim
```

asciinema(1) Allgemeines Befehlshandbuch asciinema(1)

NAME asciinema – Terminal-Sitzungsrekorder, Streamer und Player

ZUSAMMENFASSUNG asciinema [-q|–quiet] [-h|–help] [-V|–version]

BESCHREIBUNG Terminal-Sitzungsrekorder, Streamer und Player

OPTIONEN -q, –quiet Unterdrücken Sie Diagnosemeldungen und Fortschrittsanzeigen. Es werden nur Fehlermeldungen angezeigt.

-h, –help Hilfe drucken (Zusammenfassung mit '-h' anzeigen)

-V, –version Druckversion

UNTERBEFEHLE asciinema-record(1) Zeichnen Sie eine Terminalsitzung auf

asciinema-stream(1) Streamen Sie eine Terminalsitzung

asciinema-session(1) Zeichnen Sie eine Terminalsitzung auf und streamen Sie sie

asciinema-play(1) Spielen Sie eine Terminalsitzung ab

asciinema-upload(1) Laden Sie eine Aufnahme auf einen Asciinema-Server hoch

asciinema-auth(1) Authentifizieren Sie diese CLI mit einem Asciinema-Serverkonto

asciinema-cat(1) Mehrere Aufnahmen verketten

asciinema-convert(1) Konvertieren Sie eine Aufnahme in ein anderes Format

VERSION v3.0.0

AUTOREN Marcin Kulik m@ku1ik.com

asciinema 3.0.0 asciinema(1)

Vorraussetzungen

System Details Report

Report details

- **Date generated:** 2026-08-09 16:10:47

Hardware Information:

- **Hardware Model:** Dell Inc. OptiPlex 790
- **Memory:** 8.0 GiB
- **Processor:** Intel® Core™ i5-2500S × 4
- **Graphics:** Intel® HD Graphics 2000 (SNB GT1)
- **Disk Capacity:** 250.1 GB

Software Information:

- **Firmware Version:** A16
- **OS Name:** Fedora Linux 44 (Workstation Edition)
- **OS Build:** (null)
- **OS Type:** 64-bit
- **GNOME Version:** 50
- **Windowing System:** Wayland
- **Kernel Version:** Linux 7.1.7-200.fc44.x86_64

Hugo Container Workflow (Fedora 43 / Podman)

1. Containerfile für die lokale Entwicklung

Erstellt in `my-site/Containerfile`:

```
FROM floryn90/hugo:ext

WORKDIR /src
COPY . /src
EXPOSE 1313

CMD ["server", "-D", "--bind", "0.0.0.0"]
```

2. Image bauen

```
podman build -t mein-hugo-blog .
```

3. Dev-Server mit Live-Reload starten

```bash

podman run --rm -it --user 0:0 -p 1313:1313 -v $(pwd):/src:Z mein-hugo-blog
```

URL: http://localhost:1313/my-site/
Fix für Lock-Dateien: Falls ein Permission Error auftritt: rm -f .hugo_build.lock

---

Schritt 2: Den funktionierenden Stand in Git sichern

Jetzt frieren wir den Zustand im Git-Repository ein:

```
# 1. Status prüfen
git status

# 2. Containerfile und geänderte Dateien stagen
git add Containerfile hugo.toml commands.md

# 3. Commit erstellen
git commit -m "feat: Containerfile für lokale Hugo-Entwicklung hinzugefügt"
```

```
# 1. In das Hugo-Projektverzeichnis wechseln
cd my-site

# 2. .gitignore anlegen (damit der generierte public/-Ordner ignoriert wird)
vim .gitignore
```

Füge in die .gitignore ein:
Plaintext

```
public/
resources/
.hugo_build.lock
```

Anschließend das Repository initialisieren und den ersten Commit erstellen:
Bash

```
git init
git branch -M main
git add .
git commit -m "initial: Hugo Blog mit PaperMod und Containerfile"

vim .gitignore
```

Füge in die .gitignore ein:
Plaintext

```
my-site/public/
my-site/resources/
my-site/.hugo_build.lock
```

Anschließend alles committen:
Bash

```
git init
git branch -M main
git add .
git commit -m "initial: Hugo-Projekt, Dokus und Asciinema-Casts"
```

2. Produktions-Build mit Caddy (Multi-Stage)

Containerfile.prod
Erstellt für schlanke Auslieferung via Caddy mit angepasster BaseURL:

```dockerfile
FROM floryn90/hugo:ext AS builder
WORKDIR /src
COPY . /src
RUN ["hugo", "--minify", "--noBuildLock", "-b", "http://localhost:8080/"]

FROM caddy:alpine
COPY --from=builder /src/public /usr/share/caddy
EXPOSE 80
```

```
# Image baueni

podman build -f Containerfile.prod -t mein-hugo-blog:prod .

# Caddy-Container auf Port 8080 starten

podman run --rm -d -p 8080:80 --name blog-prod mein-hugo-blog:prod

# Container stoppen

podman stop blog-prod
```

```
Hinweise zur Fehlerbehebung

    CSS/Assets laden nicht (weiße Seite): Sicherstellen, dass -b im hugo-Befehl der
     ↪  tatsächlichen URL/Port entspricht.

    Permission Errors beim Bauen: .containerignore muss public/ und .hugo_build.lock enthalten.


Speichere die Datei mit **`:wq`**.

---

### Schritt 3: Auch die geänderte Doku committen
```

git add commands.md git commit -m “docs: Produktions-Workflow mit Caddy und BaseURL-Fix in commands.md ergänzt”

```
Damit ist dein gesamtes Setup - sowohl für die lokale Entwicklung als auch für
den produktiven Container - vollumfänglich versioniert und dokumentiert!

---
```

[stephan@dell-optiplex-kueche commands]$ gh gist create commands.md -d “Hugo, Podman & Caddy Workflow auf Fedora” -w - Creating gist commands.md [X] Created secret gist commands.md Opening https://gist.github.com/bac-stephan/e0e2ae84eb9195d51462639ae27ec7e3 in your browser. [stephan@dell-optiplex-kueche commands]$

“‘